Transform Your Future with
"HELLO WORLD!"
INSTITUTE

Why learn with us ?

Training From Experts

One - on – One Guidance

100% Practical

Real-Time Projects & Case Studies

Placement Preparation Support

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Contact :- 8085896740 @ hello_world_institute Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Tuples

A collection of ordered and immutable objects is known as a tuple. Tuples and lists are similar as they both are sequences. Though, tuples and lists are different because we cannot modify tuples, although we can modify lists after creating them, and also because we use parentheses to create tuples while we use square brackets to create lists.

Placing different values separated by commas and enclosed in parentheses forms a tuple.

A tuple is a collection which is ordered and **unchangeable**

```
tup1 = ('p hysics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

.

Tuple Items

- Tuple items are ordered, unchangeable, and allow duplicate values.
- Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Eg:

```
tuple = ("apple", "banana", "cherry", "apple", "cherry")
print(tuple)
```

Tuple Length

To determine how many items a tuple has, use the len() function:

Eg:

```
tuple = ("apple", "banana", "cherry")
print(len(tuple))
```

Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

Eg: One item tuple, remember the comma:

```
tuple = ("apple",)
print(type(tuple))

#NOT a tuple
tuple = ("apple")
print(type(tuple))
```

Tuple Items - Data Types

Tuple items can be of any data type:

Eg: String, int and boolean data types:

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

A tuple can contain different data types:

Eg: A tuple with strings, integers and boolean values:

```
tuple1 = ("abc", 34, True, 40, "male")
```

type()

From Python's perspective, tuples are defined as objects with the data type 'tuple':

```
<class 'tuple'>
```

Eg:

```
mytuple = ("apple", "banana", "cherry")
print(type(mytuple))
```

The tuple() Constructor

It is also possible to use the tuple() constructor to make a tuple.

Eg : Using the tuple() method to make a tuple:

```
tuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(tuple)
```

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets:

Eg:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

Note: The first item has index 0.

Negative Indexing

Negative indexing means start from the end, -1 refers to the last item, -2 refers to the second last item etc.

Eg:

```
tuple1 = ("apple", "banana", "cherry")

print(tuple1[-1])
```

Range of Indexes

- You can specify a range of indexes by specifying where to start and where to end the range.
- When specifying a range, the return value will be a new tuple with the specified items.

Eg:

```
tuple1 = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")

print(tuple1[2:5])
```

Eg:

```
tuple1 = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")

print(tuple1[:4])
```

Example

This example returns the items from "cherry" and to the end:

```
tup = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")

print(tup[2:])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the tuple:

Eg:

```
tup = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")

print(tup[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a tuple use the in keyword:

Eg:

```
tup = ("apple", "banana", "cherry")
 if "apple" in tup:

  print("Yes, 'apple' is in the fruits tuple")
```

Update Tuples

Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created.But there are some workarounds.

Change Tuple Values

- Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called.
- But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

Eg: Convert the tuple into a list to be able to change it:

```
x = ("apple", "banana", "cherry")

y = list(x)
y[1] = "kiwi"
x = tuple(y)
print(x)
```

Add Items

Since tuples are immutable, they do not have a build-in append() method, but there are other ways to add items to a tuple.

1. **Convert into a list**: Just like the workaround for *changing* a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

Eg: Convert the tuple into a list, add "orange", and convert it back into a tuple:

```
tuple1 = ("apple", "banana", "cherry")

y = list(tuple1)

y.append("orange")

tuple1 = tuple(y)
```

2. **Add tuple to a tuple**. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

Ex: Create a new tuple with the value "orange", and add that tuple:

```
tup = ("apple", "banana", "cherry")
 y = ("orange",)

tup += y
 print(tup)
```

Note: When creating a tuple with only one item, remember to include a comma after the item, otherwise it will not be identified as a tuple.

Remove Items

Note: You cannot remove items in a tuple.
Tuples are **unchangeable**, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

Eg :Convert the tuple into a list, remove "apple", and convert it back into a tuple:

```
tup = ("apple", "banana", "cherry")

y = list(tup)
y.remove("apple")
tup = tuple(y)
```

Or you can delete the tuple completely:

Ex: The del keyword can delete the tuple completely:

```
tuple1= ("apple", "banana", "cherry")

del tuple1
print(tuple1) #this will raise an error because the tuple no longer exists
```

Unpack Tuples

Unpacking a Tuple

When we create a tuple, we normally assign values to it. This is called "packing" a tuple:
Eg: Packing a tuple:

```
fruits = ("apple", "banana", "cherry")
```

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking":

Eg : Unpacking a tuple:

```
fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits
print(green)
print(yellow)
print(red)
```

Note: The number of variables must match the number of values in the tuple, if not, you must use an asterisk to collect the remaining values as a list.

Using Asterisk*

If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list:

Eg: Assign the rest of the values as a list called "red":

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, *red) = fruits
print(green)
print(yellow)
print(red)
```

If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

Eg : Add a list of values the "tropic" variable:

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")

(green, *tropic, red) = fruits
print(green)
print(tropic)
print(red)
```

Loop Tuples : You can loop through the tuple items by using a for loop.

Eg : Iterate through the items and print the values:

```
tuple1 = ("apple", "banana", "cherry")

for x in tuple1:
  print(x)
```

Loop Through the Index Numbers

You can also loop through the tuple items by referring to their index number. Use the range() and len() functions to create a suitable iterable.

Eg : Print all items by referring to their index number:

```
tuple1 = ("apple", "banana", "cherry")

for i in range(len(tuple1)):
  print(tuple1[i])
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Using a While Loop

- Use the len() function to determine the length of the tuple, then start at 0 and loop your way through the tuple items by referring to their indexes.
- Remember to increase the index by 1 after each iteration

Eg:

```
tuple1 = ("apple", "banana", "cherry")

i = 0
while i<len(tuple1):
  print(tuple1[i])
  i = i + 1
```

Join Tuples

1. **Join Two Tuples :** To join two or more tuples you can use the + operator:

Eg:

```
tuple1 = ("a", "b" , "c")

tuple2 = (1, 2, 3)


tuple3 = tuple1 + tuple2

print(tuple3)
```

2. **Multiply Tuples :** If you want to multiply the content of a tuple a given number of times, you can use the * operator:

Eg:

```
fruits = ("apple", "banana", "cherry")

mytuple = fruits * 2
 print(mytuple)
```

OFFLINE CLASSES

ONLINE LECTURE

RECORDED ACCESS

Tuple Methods

Python has two built-in methods that you can use on tuples :

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found